

Demo 01 — MLOps Fundamentals

Labs 1–4 Practical Summary & EC2 Lab Record

Environment: AWS EC2 Ubuntu | Python 3.12 host | Virtual environment | Docker

Purpose: A reusable reference of what was built, what each lab demonstrates, and the successful validation performed during the Demo 01 hands-on labs.

Lab	Topic	What we achieved	Result
01	MLflow Tracking	Trained a Random Forest model and tracked an ML run with MLflow.	PASS
02	ML Lifecycle & Registry	Built ingest → validate → train → evaluate → quality gate → model registration	PASS + FAIL test
03	Docker Reproducibility	Built a pinned Python environment in Docker and compared it with the EC2 host.	PASS.
04	Pipeline Automation	Used Make dependencies to automate ingest → train → evaluate and incremental execution.	PASS.

Lab 01 — MLflow Experiment Tracking

Objective: Move from a manually trained ML model to an experiment whose run information is persisted and visible through MLflow.

What we did

- Created a Python virtual environment on the EC2 instance.
- Installed the required ML/MLOps packages.
- Trained a Random Forest classifier using the demo dataset.
- Verified the manual training result with **Accuracy: 1.0**.
- Added MLflow tracking to the training workflow.
- Created the MLflow experiment **intro-demo**.
- Logged the accuracy metric and the trained model artifact.

Important validation

The tracked training completed successfully with **Logged run with accuracy: 1.0** and exit code 0. An MLflow database and **mlruns** directory were created in the demo workspace.

Concept learned

MLflow Tracking gives us a persistent record of an ML run: metrics, model artifacts, and experiment/run metadata. This is the first step toward repeatable MLOps instead of one-off model training.

Lab 01 flow

Dataset → Train Random Forest → Evaluate → MLflow Experiment → Log Accuracy → Log Model

Lab 02 — ML Lifecycle, Quality Gate & Model Registry

Objective: Build a small end-to-end lifecycle where data is ingested and validated, a model is trained, a quality threshold is checked, and only a passing model is registered.

What we did

- Loaded **178 rows** of data.
- Passed data validation checks.
- Trained the model with **accuracy=1.000**.
- Implemented an evaluation threshold/quality gate.
- Registered the model as **wine-lifecycle-demo**.
- Successfully created model version **2** after a subsequent successful run.

Quality-gate PASS test

acc=1.000 vs threshold=0.99 → PASS. The model was registered successfully.

Quality-gate FAIL test

We deliberately changed the threshold to **1.001** and reran the pipeline. The output was **acc=1.000 vs threshold=1.001 → FAIL**, followed by **Model did not meet quality bar — not registered**. This verified that the gate prevents an unsuccessful model from entering the registry.

Concept learned

A production-style ML lifecycle should not automatically register every trained model. A quality gate provides an explicit control point between evaluation and model registration.

Lab 03 — Reproducible Environment with Docker

Objective: Remove dependency on the host Python/package versions by defining a reproducible environment in a Docker image.

What we created

Created **env_setup_demo** containing:

```
requirements.txt
• scikit-learn==1.5.2
• pandas==2.2.3
• mlflow==2.16.2

Dockerfile
• Base image: python:3.11-slim
• Installs the pinned requirements
• Copies check_env.py
• Runs the environment check
```

Build result

Docker successfully built and tagged the image as **env-demo:latest**.

Runtime validation

Running **docker run --rm env-demo** produced:

```
Python: 3.11.15
scikit-learn: 1.5.2
pandas: 2.2.3
```

Host vs container

The EC2 host had Python **3.12.3**, scikit-learn **1.9.0**, and pandas **2.3.3**. The container independently supplied Python **3.11.15**, scikit-learn **1.5.2**, and pandas **2.2.3**.

Concept learned

Docker packages the application with a defined runtime and dependency set. This makes the ML environment reproducible across machines and reduces the classic “works on my machine” problem.

Lab 04 — Pipeline Automation with Make

Objective: Replace manual execution order with a dependency-aware pipeline that can be run using one command.

Stages created

- **ingest.py** — fetches demo data and writes **data.csv**.
- **train.py** — reads **data.csv** and creates **model.txt**.
- **evaluate.py** — reads **model.txt** and reports evaluation accuracy.
- **Makefile** — defines dependencies between these stages.

Dependency graph

```
ingest.py → data.csv → train.py → model.txt → evaluate.py
```

Manual-order problem demonstrated

Running **python src/evaluate.py** before the required upstream files exist fails because **model.txt** has not been created. This demonstrates why manually remembering pipeline order is error-prone.

Automated run

After installing GNU Make, **make clean** removed generated files and **make** successfully executed **ingest → train → evaluate** in the correct order. The final evaluation reported **accuracy=0.95**.

Incremental execution

Running **make** again is expected to recognize that the target is already up to date, avoiding unnecessary reruns when the inputs have not changed.

Concept learned

A pipeline is a dependency graph. Automation tools can determine what must run and what can be skipped, making ML workflows more repeatable and efficient.

Demo 01 — Overall Learning Path

Lab 01: Track ML experiments → Lab 02: Control model lifecycle → Lab 03: Reproduce the runtime → Lab 04: Automate the workflow

Final Demo 01 outcome: We progressed from a manually trained model to a tracked, quality-gated, registered, containerized, and dependency-aware ML workflow.